

角色 B 后端数据库与接口文档（草案）

适用角色：成员 B

目标：在不破坏现有 `/analyze` SSE 链路的前提下，为分析记录、事件流、工具调用、总结结果、会话追问和历史恢复提供 SQLite 持久化与查询接口。

1. 文档范围

本文档只定义 B 负责的后端接口与数据边界，不修改 Agent 核心推理逻辑。

覆盖内容：

- 1. 现有接口的兼容方式
- 2. SQLite 首版数据库表结构
- 3. 分析会话、事件流、总结、对话、skill 的接口
- 4. SSE 事件与数据库落库映射

不覆盖内容：

- 1. Agent 内部辩论策略
- 2. 领域识别与 skill 选择逻辑
- 3. 前端 UI 细节

2. 当前已存在的接口

仓库当前后端入口位于 `backend/main.py`，已存在以下接口：

方法	路径	说明
GET	<code>/health</code>	健康检查，返回后端和 Agent 初始化状态
GET / POST	<code>/analyze</code>	以 SSE 形式输出文章分析事件
POST	<code>/v1/run</code>	评估平台兼容入口，SSE 输出与 <code>/analyze</code> 一致
GET	<code>/datasets/{dataset_id}</code>	数据集加载接口

现有 SSE 事件定义在 `agent/models.py`，当前包含：

- plan



Agent 正在扫描全网核查此页面...

- thinking
- tool_call
- annotation
- error
- status
- done

兼容原则

1. 现有事件字段保持兼容，前端旧逻辑不能被破坏。
2. 新增功能采用“向后兼容扩展”，即新增可选事件或新增接口，不直接改义已有字段。
3. 现有 `/analyze` 和 `/v1/run` 继续可用，新的会话和历史能力通过新接口叠加。

3. 首版数据库选型

首版使用 SQLite，原因是：

1. 部署简单，适合课程项目快速落地。
2. 不依赖额外数据库服务，便于本地联调和演示。
3. 后续若需要迁移 PostgreSQL/MySQL，可将 ORM/DAO 层替换而不影响 API 结构。

3.1 推荐技术栈

- FastAPI
- SQLAlchemy 2.x
- SQLite
- Alembic（可选，后续做迁移管理）

3.2 通用约定

- 所有主键使用字符串 ID，建议 `uuid4().hex` 或带前缀的业务 ID。
- 时间统一使用 UTC ISO8601 字符串存储。
- JSON 结构统一存 TEXT 或数据库原生 JSON 字段，首版 SQLite 建议 TEXT。
- SSE 原始事件和数据库事件记录必须共享同一份 payload 结构。

4. 数据表设计

下面是 B 负责的首版核心表。5



Agent 正在扫描全网核查此页面...

保持一致。

4.1 analysis_session

分析会话主表，一次网页分析对应一条记录。

字段	类型	说明
id	TEXT PK	会话 ID
article_title	TEXT	网页标题，可选
source_url	TEXT	原始页面地址，可选
article_text	TEXT	本次分析的正文快照
article_hash	TEXT	正文哈希，用于去重和恢复
domain	TEXT	领域标签，如 <code>medical / finance / technology / news_policy / general</code>
skill_id	TEXT	使用的 skill ID，可为空
status	TEXT	<code>pending / running / completed / failed</code>
total_claims	INTEGER	声明总数
total_annotations	INTEGER	错误标注数
created_at	TEXT	创建时间
started_at	TEXT	开始时间
finished_at	TEXT	结束时间
error_message	TEXT	全局错误信息，可为空

4.2 claim_record

单条声明记录。

字段	类型	说明
id	TEXT PK	记录 ID
session_id	TEXT FK	所属会话
claim_id	TEXT	业务声明 ID，如 <code>c001</code>
text	TEXT	声明文本
start_offset	INT	



Agent 正在扫描全网核查此页面...

字段	类型	说明
end_offset	INTEGER	结束偏移
suspicion_score	REAL	可疑度
verdict	TEXT	pending / verified / rejected / uncertain
error_type	TEXT	错误类型，可为空
confidence	REAL	置信度
reasoning	TEXT	推理摘要
evidence_urls	TEXT	JSON 数组字符串
created_at	TEXT	创建时间
updated_at	TEXT	更新时间

4.3 event_record

事件流归档表，保存前端 SSE 输出内容。

字段	类型	说明
id	TEXT PK	记录 ID
session_id	TEXT FK	所属会话
seq	INTEGER	会话内递增序号
event_type	TEXT	status / plan / thinking / tool_call / annotation / error / summary / debate / chat / done
claim_id	TEXT	关联声明，可为空
payload	TEXT	事件完整 JSON
created_at	TEXT	创建时间

4.4 tool_call_record

工具调用明细表。

字段	类型	说明
id	TEXT PK	记录 ID

字段	类型	说明
session_id	TEXT FK	所属会话
claim_id	TEXT	关联声明，可为空
tool_name	TEXT	工具名
tool_input	TEXT	输入
tool_output	TEXT	输出
status	TEXT	success / error
latency_ms	INTEGER	耗时
error_message	TEXT	异常信息，可为空
created_at	TEXT	创建时间

4.5 summary_record

分析总结表，用于侧边栏展示与历史恢复。

字段	类型	说明
id	TEXT PK	记录 ID
session_id	TEXT FK	所属会话
overall_conclusion	TEXT	总体结论
total_claims	INTEGER	声明总数
total_errors	INTEGER	错误数
error_breakdown	TEXT	JSON 对象字符串
representative_evidence	TEXT	JSON 数组字符串
confidence	REAL	总结置信度
summary_payload	TEXT	总结事件完整 JSON
created_at	TEXT	创建时间

4.6 chat_message

对话消息表，用于追问与历史恢复。



Agent 正在扫描全网核查此页面...

字段	类型	说明
id	TEXT PK	消息 ID
session_id	TEXT FK	所属会话
role	TEXT	user / assistant / system
content	TEXT	消息内容
related_claim_id	TEXT	关联声明，可为空
message_type	TEXT	question / answer / recheck / note
metadata	TEXT	JSON 字符串
created_at	TEXT	创建时间

4.7 skill_record

skill 配置表，保存领域化提示词与工具白名单。

字段	类型	说明
id	TEXT PK	skill ID
skill_name	TEXT	skill 名称
domain	TEXT	适用领域
system_prompt	TEXT	system prompt 模板
claim_filter_rule	TEXT	声明筛选规则
allowed_tools	TEXT	JSON 数组字符串
output_contract	TEXT	输出格式约束
version	TEXT	skill 版本
is_active	INTEGER	是否启用
created_at	TEXT	创建时间
updated_at	TEXT	更新时间



5. 对外接口设计

5.1 健康检查

GET /health

响应

```
{
  "status": "ok",
  "agent_initialized": true,
  "db_initialized": true
}
```

说明

- 兼容现有返回结构。
- 建议新增 `db_initialized` 字段，用于确认 SQLite 和表结构已准备好。

5.2 发起分析

POST /analyze

请求体

```
{
  "article_text": "...",
  "article_title": "可选",
  "source_url": "可选",
  "session_id": "可选，恢复既有会话时传入",
  "skill_id": "可选，强制指定 skill 时传入"
}
```

查询参数

- `text`：与当前实现兼容，优先级低于请求体 `article_text`

响应

SSE 流，事件类型与 [agent/models.py](#) 保持一致，并扩展新的总结/辩论/对话事件。

兼容要求

- 1. 旧客户端只处理 plan / thinking / tool_call / annotation / error / done 时不应报错。
- 2. 新客户端可额外消费 summary / debate / chat 。

5.3 评估平台入口

POST /v1/run

请求体

```
{
  "article_text": "...",
  "article_title": "可选",
  "source_url": "可选",
  "session_id": "可选"
}
```

响应

与 /analyze 一致，仍返回 SSE。

5.4 会话创建

POST /api/v1/sessions

用途

创建一个新的分析会话，供后续分析、追问和历史恢复使用。

请求体

```
{
  "article_title": "可选",
  "source_url": "可选",
  "article_text": "可选",
  "domain": "可选",
  "skill_id": "可选"
}
```


响应

```
{
  "session_id": "sess_01H...",
  "status": "pending",
  "created_at": "2026-06-07T12:00:00Z"
}
```

5.5 会话列表

GET /api/v1/sessions

查询参数

- limit：默认 20
- offset：默认 0
- status：可选，按状态过滤
- domain：可选，按领域过滤

响应

```
{
  "items": [
    {
      "session_id": "sess_01H...",
      "article_title": "...",
      "domain": "finance",
      "status": "completed",
      "total_claims": 12,
      "total_annotations": 3,
      "created_at": "..."
    }
  ],
  "total": 128
}
```

5.6 会话详情

GET /api/v1/sessions/{session

响应

返回会话主表、总结、声明统计、最近事件摘要。

```
{
  "session": {
    "id": "sess_01H...",
    "status": "completed",
    "domain": "general"
  },
  "summary": {
    "overall_conclusion": "..."
  },
  "claims": [],
  "recent_events": []
}
```

5.7 会话事件历史

GET /api/v1/sessions/{session_id}/events

查询参数

- limit：默认 100
- after_seq：可选，只返回该序号之后的事件
- event_type：可选，按类型过滤

响应

```
{
  "items": [
    {
      "seq": 1,
      "event_type": "plan",
      "payload": {}
    }
  ]
}
```

5.8 单条声明详情

GET /api/v1/sessions/{session_id}/claims/{claim_id}

响应

返回单条声明、相关工具调用、辩论片段和最终结论。

```
{
  "claim": {
    "claim_id": "c001",
    "text": "...",
    "error_type": "factual_error",
    "confidence": 0.92
  },
  "tool_calls": [],
  "debate": [],
  "annotation": {}
}
```

5.9 会话总结

GET /api/v1/sessions/{session_id}/summary

响应

```
{
  "session_id": "sess_01H...",
  "overall_conclusion": "...",
  "total_claims": 12,
  "total_errors": 3,
  "error_breakdown": {
    "factual_error": 2,
    "unsupported_claim": 1
  },
  "representative_evidence": [
    {
      "claim_id": "c001",
      "text": "...",
      "evidence_urls": ["..."]
    }
  ],
  "confidence": 0.87
}
```

5.10 发送追问

POST /api/v1/sessions/{session_id}/chat

请求体

```
{
  "message": "为什么这条被判错？",
  "related_claim_id": "c001",
  "mode": "explain"
}
```

参数说明

- mode 可选值建议为：explain / recheck / evidence / summary
- related_claim_id 可为空，表示对整个会话追问

响应

```
{
  "message_id": "msg_01H...",
  "session_id": "sess_01H...",
  "status": "accepted"
}
```

5.11 拉取对话历史

GET /api/v1/sessions/{session_id}/chat

查询参数

- limit：默认 50
- before_id：可选，用于分页

响应

```
{
  "items": [
    {
      "id": "msg_01H...",
      "role": "user",
      "content": "为什么这条被判错？",
      "related_claim_id": "c001",
      "created_at": "..."
    }
  ]
}
```

5.12 重新验证

POST /api/v1/sessions/{session_id}/recheck

用途

在保留原会话上下文的前提下，对指定声明或整篇文章发起重新验证。

请求体

```
{
  "claim_id": "c001",
  "reason": "用户追问后要求重新核验",
  "skill_id": "可选"
}
```

响应

```
{
  "session_id": "sess_01H...",
  "claim_id": "c001",
  "status": "queued"
}
```

5.13 skill 查询与管理

建议预留以下接口，供 D 和 B 联调：

- GET /api/v1/skills
- GET /api/v1/skills/{skill_id}
- POST /api/v1/skills
- PUT /api/v1/skills/{skill_id}

首版可以只实现查询，不强制实现在线编辑。

6. SSE 事件入库映射

SSE 输出和数据库必须一一对应。建议规则如下：

事件类型	是否入库到 event_record	是否同步写其他表
status	是	否
plan	是	是，更新 analysis_session.total_claims 、 claim_record 初始数据
thinking	是	

事件类型	是否入库到 event_record	是否同步写其他表
tool_call	是	是，写 tool_call_record
annotation	是	是，更新 claim_record、analysis_session.total_annotations
error	是	是，更新会话或声明错误状态
summary	是	是，写 summary_record
debate	是	可选，若需要完整辩论链则同步归档
chat	是	是，写 chat_message
done	是	是，更新会话状态

建议实现顺序

- 1. 先保证所有事件都能写入 event_record。
- 2. 再补 tool_call_record、claim_record、summary_record。
- 3. 最后补会话和对话接口。

7. 错误码建议

HTTP 状态	场景
400	请求参数缺失或格式不合法
404	会话、声明或 skill 不存在
409	会话状态冲突，例如已完成会话不可重复覆盖
500	数据库异常或未捕获的系统异常

建议统一错误响应格式：

```
{
  "error": {
    "code": "SESSION_NOT_FOUND",
    "message": "session not found"
  }
}
```

8. 实现建议

8.1 接口层拆分

建议 backend/ 下拆成：

```
backend/  
├─ main.py  
├─ db/  
│   ├── session.py  
│   ├── models.py  
│   ├── repository.py  
│   └─ init_db.py  
├─ schemas/  
│   ├── session.py  
│   ├── chat.py  
│   └─ skill.py  
└─ services/  
    ├── event_store.py  
    ├── session_service.py  
    └─ chat_service.py
```

8.2 最小可交付版本

如果时间紧，B 的最小闭环建议是：

- 1. /analyze 接口增加 session 记录与事件入库。
- 2. analysis_session、event_record、tool_call_record、summary_record 四张表先落地。
- 3. 提供
GET /api/v1/sessions、GET /api/v1/sessions/{session_id}、
GET /api/v1/sessions/{session_id}/summary 三个查询接口。
- 4. 再补 chat 和 recheck。

9. 与其他成员的接口对齐点

对 A

- 1. 需要 A 在事件里明确 summary
- 2. 需要 A 在 done 之前给出最

对 C

1. 需要 C 约定 summary 卡片和会话区读取的接口字段。
2. 需要 C 处理 SSE 与历史接口的字段一致性。

对 D

1. skill_record 的 schema 需与 D 的 skill 设计同步。
2. domain 标签枚举需统一。

对 E

1. 工具调用结果和失败信息需稳定写入 tool_call_record。
2. 如后续评估平台需要过程指标，event_record 和 claim_record 需保留足够细节。

10. 推荐本阶段冻结字段

为避免联调时反复改表，建议先冻结以下字段：

1. analysis_session.id
2. analysis_session.domain
3. analysis_session.status
4. claim_record.claim_id
5. claim_record.error_type
6. event_record.event_type
7. event_record.payload
8. summary_record.error_breakdown
9. chat_message.related_claim_id
10. skill_record.allowed_tools

11. 结论

角色 B 的工作重点不是单纯“加数据库”，而是把现有 SSE 分析流变成可查询、可恢复、可追问的会话系统。

首版最重要的接口边界是：

1. /analyze 继续作为分析入口
2. event_record 负责归档所有



Agent 正在扫描全网核查此页面...

3. `analysis_session` 负责承载一次分析的状态

4. `summary_record` 和 `chat_message` 负责让前端展示总结和追问

只要这四层稳定，后续再扩展辩论、领域 skill 和更多工具时，后端接口就不会失控。